

A Technical Introduction to Actor critic algorithm

Section 1

The actor-critic algorithm (AC) is a family of reinforcement learning (RL) algorithms that combine policy-based RL algorithms such as policy gradient methods, and value-based RL algorithms such as value iteration, Q-learning, SARSA, and TD learning. An AC algorithm consists of two main components: an "actor" that determines which actions to take according to a policy function, and a "critic" that evaluates those actions according to a value function. Some AC algorithms are on-policy, some are off-policy. Some apply to either continuous or discrete action spaces. Some work in both cases.

Section 2

$$\gamma_j (R_j + \gamma V_{\pi_{\theta}}(S_{j+1}) - V_{\pi_{\theta}}(S_j))$$

$$\gamma_j (R_j + \gamma V_{\pi_{\theta}}(S_{j+1}) - V_{\pi_{\theta}}(S_j))$$
 : TD(1) learning.

Section 3

$$\gamma_j (R_j + \gamma R_{j+1} + \gamma^2 V_{\pi_{\theta}}(S_{j+2}) - V_{\pi_{\theta}}(S_j))$$

$$\gamma_j (R_j + \gamma R_{j+1} + \gamma^2 V_{\pi_{\theta}}(S_{j+2}) - V_{\pi_{\theta}}(S_j))$$
 : TD(2) learning.

Section 4

Variants Asynchronous Advantage Actor-Critic (A3C): Parallel and asynchronous version of A2C. Soft Actor-Critic (SAC): Incorporates entropy maximization for improved exploration. Deep Deterministic Policy Gradient (DDPG): Specialized for continuous action spaces.

Section 5

$$\gamma_j \sum_{n=1}^{\infty} \lambda^{n-1} (R_{j+n} - V_{\pi_{\theta}}(S_{j+n}))$$

$$\gamma_j \sum_{n=1}^{\infty} \lambda^{n-1} (R_{j+n} - V_{\pi_{\theta}}(S_{j+n}))$$
 : TD(λ) learning, also known as GAE (generalized advantage estimate). This is obtained by an exponentially decaying sum of the TD(n) learning terms.

Section 6

Actor The actor uses a policy function $\pi(a|s)$, while the critic estimates either the value function $V(s)$, the action-value Q-function $Q(s,a)$, the advantage function $A(s,a)$, or any combination thereof. The actor is a parameterized function π_{θ} , where θ are the parameters of the actor. The actor takes as

argument the state of the environment s and produces a probability distribution $\pi_{\theta}(a|s)$. If the action space is discrete, then $\sum_a \pi_{\theta}(a|s) = 1$. If the action space is continuous, then $\int_a \pi_{\theta}(a|s) da = 1$. The goal of policy optimization is to improve the actor. That is, to find some θ that maximizes the expected episodic reward $J(\theta)$: $J(\theta) = E_{\pi_{\theta}}[\sum_{t=0}^T \gamma^t r_t]$ where γ is the discount factor, r_t is the reward at step t , and T is the time-horizon (which can be infinite). The goal of policy gradient method is to optimize $J(\theta)$ by gradient ascent on the policy gradient $\nabla_{\theta} J(\theta)$. As detailed on the policy gradient method page, there are many unbiased estimators of the policy gradient: $\nabla_{\theta} J(\theta) = E_{\pi_{\theta}}[\sum_{0 \leq j \leq T} \nabla_{\theta} \ln \pi_{\theta}(A_j|S_j) \cdot \Psi_j | S_0 = s_0]$ where Ψ_j is a linear sum of the following:

Section 7

Critic In the unbiased estimators given above, certain functions such as $V_{\pi_{\theta}}$, $Q_{\pi_{\theta}}$, $A_{\pi_{\theta}}$ appear. These are approximated by the critic. Since these functions all depend on the actor, the critic must learn alongside the actor. The critic is learned by value-based RL algorithms. For example, if the critic is estimating the state-value function $V_{\pi_{\theta}}(s)$, then it can be learned by any value function approximation method. Let the critic be a function approximator $V_{\phi}(s)$ with parameters ϕ . The simplest example is TD(1) learning, which trains the critic to minimize the TD(1) error: $\delta_i = R_{i+1} + \gamma V_{\phi}(S_{i+1}) - V_{\phi}(S_i)$. The critic parameters are updated by gradient descent on the squared TD error: $\phi \leftarrow \phi - \alpha \nabla_{\phi} (\delta_i)^2 = \phi + \alpha \delta_i \nabla_{\phi} V_{\phi}(S_i)$ where α is the learning rate. Note that the gradient is taken with respect to the ϕ in $V_{\phi}(S_i)$ only, since the ϕ in $\gamma V_{\phi}(S_{i+1})$ constitutes a moving target, and the gradient is not taken with respect to that. This is a common source of error in implementations that use automatic differentiation, and requires "stopping the gradient" at that point. Similarly, if the critic is

estimating the action-value function $Q_{\pi_{\theta}}$, then it can be learned by Q-learning or SARSA. In SARSA, the critic maintains an estimate of the Q-function, parameterized by ϕ , denoted as $Q_{\phi}(s, a)$. The temporal difference error is then calculated as $\delta_i = R_{i+1} + \gamma Q_{\theta}(S_{i+1}, A_{i+1}) - Q_{\theta}(S_i, A_i)$. The critic is then updated by $\theta \leftarrow \theta + \alpha \delta_i \nabla_{\theta} Q_{\theta}(S_i, A_i)$. The advantage critic can be trained by training both a Q-function $Q_{\phi}(s, a)$ and a state-value function $V_{\phi}(s)$, then let $A_{\phi}(s, a) = Q_{\phi}(s, a) - V_{\phi}(s)$. Although, it is more common to train just a state-value function $V_{\phi}(s)$, then estimate the advantage by $A_{\phi}(S_i, A_i) \approx \sum_{j=0}^{n-1} \gamma^j R_{i+j} + \gamma^n V_{\phi}(S_{i+n}) - V_{\phi}(S_i)$. Here, n is a positive integer. The higher n is, the more lower is the bias in the advantage estimation, but at the price of higher variance. The Generalized Advantage Estimation (GAE) introduces a hyperparameter λ that smoothly interpolates between Monte Carlo returns ($\lambda = 1$, high variance, no bias) and 1-step TD learning ($\lambda = 0$, low variance, high bias). This hyperparameter can be adjusted to pick the optimal bias-variance trade-off in advantage estimation. It uses an exponentially decaying average of n-step returns with λ being the decay strength.